

CS4234: Optimization Algorithms
Semester 1, AY 2026-27
School of Computing, NUS

Problem Set for Tutorial 1

Question 1: You are given an undirected graph $G = (V, E)$. The goal of this problem is to remove a minimum number of edges such that the residual graph contains no triangles. (I.e., there is no set of three vertices (a, b, c) such that edges (a, b) , (b, c) , and (a, c) are all in the residual graph.)

Give a 3-approximation algorithm that runs in polynomial time. (That is, it should guarantee that there are no triangles, and that the number of edges removed is within a factor of 3 of optimal.)

Question 2: In the first lecture, we hinted at the possibility of a *fully polynomial time approximation scheme* (FPTAS) for the *Knapsack problem* (that is, for any $0 < \epsilon < 1$, there is an algorithm that outputs a solution with value at least $(1 - \epsilon)$ times that attained by an optimal solution and running time $\text{poly}(n/\epsilon)$).

We are given n items with item i having an integral weight w_i and an integral value v_i , and a capacity W . We want to output a subset of the n items which has total weight lesser or equal to W with maximum sum value, i.e., output $S \subseteq \{1, 2, \dots, n\}$ such that $\sum_{i \in S} w_i \leq W$ which maximizes $\sum_{i \in S} v_i$. Let $V = \max_i v_i$.

Question 2A: Design an algorithm that runs in time $O(n^2V)$. [**Hint:** Think dynamic programming.]

Question 2B: Notice that if $V = O(\text{poly}(n/\epsilon))$, then the algorithm runs in poly-time. Deduce an FPTAS for the *Knapsack problem*. [**Hint:** Divide each v_i by some term depending on the approximation term ϵ and V , before running the algorithm in part A.]

Question 3: Given a graph $G = (V, E)$, an *independent set* is a subset of vertices $I \subseteq V$ such that no two nodes in I are neighbors. There is a close connection between finding a minimum vertex cover and a maximum independent set. Assume that **OPT-VC** is an algorithm for finding a minimum vertex cover. Then the following is an algorithm for finding a maximum independent set:

```
OPT-IS(V, E)
  C = OPT-VC(V, E)
  I = V \setminus C
  return I
```

That is, if C is a minimum vertex cover, then $V \setminus C$ is a maximum independent set.

Now, assume that **Greedy-Match-VC** is the 2-approximation algorithm for vertex cover that we studied in class (lecture 2). Consider the following algorithm for the independent set:

```
Greedy-IS(V, E)
  C = Greedy-Match-VC(V, E)
  I = V \setminus C
  return I
```

Give an example graph where the approximation ratio for this algorithm is as bad as possible. That is, draw a graph where the maximum independent set is very large, while the independent set returned by the **Greedy-IS** algorithm is very small.

Notice what this demonstrates: even though it is easy to translate an *optimal* algorithm for vertex cover into an *optimal* algorithm for independent set, it is very hard to turn an *approximation* algorithm for the vertex cover into an *approximation* algorithm for the independent set.